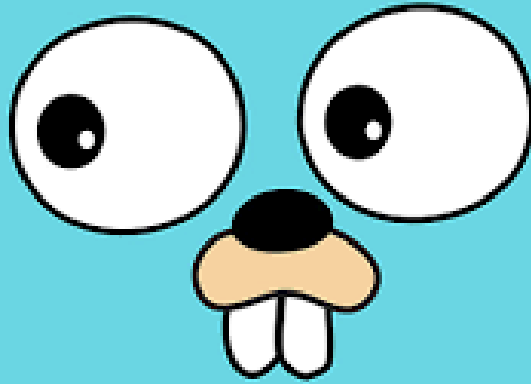




Working with Databases



Discussion Points

- Introduction to SQL in Go
- Using database package



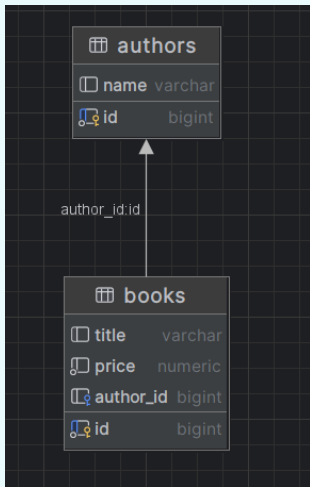


Introduction to SQL in Go

What is SQL?

Structured Query Language (SQL) is a domain-specific language used to work with relational databases. There are many dialects of SQL, but for this topic, we are focusing on **PostgreSQL**, a widely used open-source dialect.

Query	Usage	Example
CREATE TABLE	Creates a new database table	<pre>create table authors (id BIGINT generated always as identity constraint authors_pk primary key, name varchar);</pre>
SELECT	Selects rows of data from a given table following additional constraints by WHERE and ORDER BY	<pre>SELECT * FROM books WHERE title = 'Pie' ORDER BY price;</pre>
INSERT	Inserts a new row of data into a given table	<pre>INSERT INTO authors (name) VALUES ('Z. Adigozalov');</pre>
UPDATE	Updates a row/set of rows of a given table that match a given constraint, specified by WHERE	<pre>UPDATE author SET name = 'New Author' WHERE id = 1;</pre>
DELETE	Deletes a row/set of rows of a given table that match a given constraint, specified by WHERE	<pre>DELETE FROM books WHERE id = 1;</pre>
WHERE	Specifies a constraint; commonly used with SELECT, UPDATE and DELETE	See above
ORDER BY	Orders the results of a SELECT by a given column/set of columns in ascending (default) or descending order	See above
RETURNING	Returns the results of a query that might not typically return results like INSERT, UPDATE, and DELETE	<pre>INSERT INTO authors (name) VALUES ('F. Farzaliyev') RETURNING *;</pre>





Introduction to SQL in Go

Create a PostgreSQL Environment?

We will use Docker Desktop to create a docker image of postgres

```
docker run --name postgres-container -e POSTGRES_USER=postgres -e POSTGRES_PASSWORD=postgres -p 5432:5432 -d postgres
```

Explanation:

- `--name postgres-container`: Names the container `postgres-container`.
- `-e POSTGRES_USER=postgres`: Sets the PostgreSQL username to postgres.
- `-e POSTGRES_PASSWORD=postgres`: Sets the PostgreSQL password to postgres.
- `-p 5432:5432`: Maps port 5432 on your Mac to port 5432 in the container.
- `-d`: Runs the container in detached mode.
- `postgres`: Uses the official PostgreSQL image from Docker Hub. It defaults to the latest version if no tag is specified.

After running this command, you can connect to the database on localhost:5432 with the credentials provided.





Introduction to SQL in Go

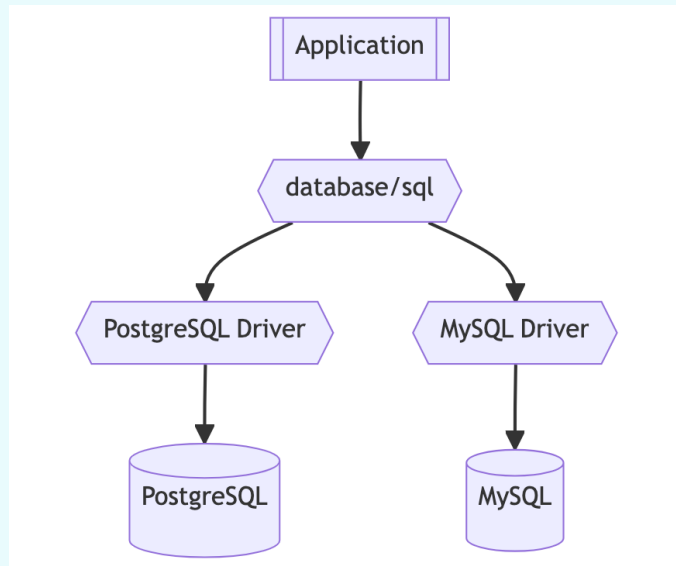
Database access in Go?

Go provides a built-in database access package (`database/sql`) which provides the necessary tools for working with SQL databases. Therefore, a firm understanding of how the `database/sql` package works is a must!

The `database/sql` package exposes a **`sql.DB`** type that provides **a layer of abstraction** over the SQL database. *However, it does not directly interact with the underlying database; that is the role of the database drivers, which we will discuss shortly.*

Through `database/sql`, uniform database access is achieved, regardless of the underlying database, which makes it easy to switch from one SQL dialect to another if necessary.

The following is an illustration of how `database/sql` works.





Introduction to SQL in Go

Database drivers in Go?

Before you can access whatever database you're using in your Go program, a corresponding database driver must be installed. This driver is the concrete link between your Go code and the database.

A list of database drivers for various SQL dialects can be found on the official [Go Wiki](#).





Using database package

Setting Up a Database Connection?

Process of establishing a database connection in Go using the database/sql package. It covers the following key points:

Overview of sql.DB:

The sql.DB object acts as a pool of database connections and is used to interact with the database. It manages the connections efficiently, including opening and reusing them as needed.

Creating a Connection String:

A connection string contains the necessary credentials and parameters to connect to a database.

Example: user=username dbname=mydb sslmode=disable

The format of the connection string varies depending on the database driver.

Opening a Database Connection:

Use sql.Open(driverName, dataSourceName) to create a sql.DB object.

The driverName is the name of the database driver (e.g., postgres, mysql).

dataSourceName is the connection string.

Error Handling:

Always check for errors when opening a connection.

Improper connection strings or driver issues can result in runtime errors.

Best Practice:

Test the connection using db.Ping() to ensure it's valid.



```
package main

import (
    "database/sql"
    _ "github.com/lib/pq"
)

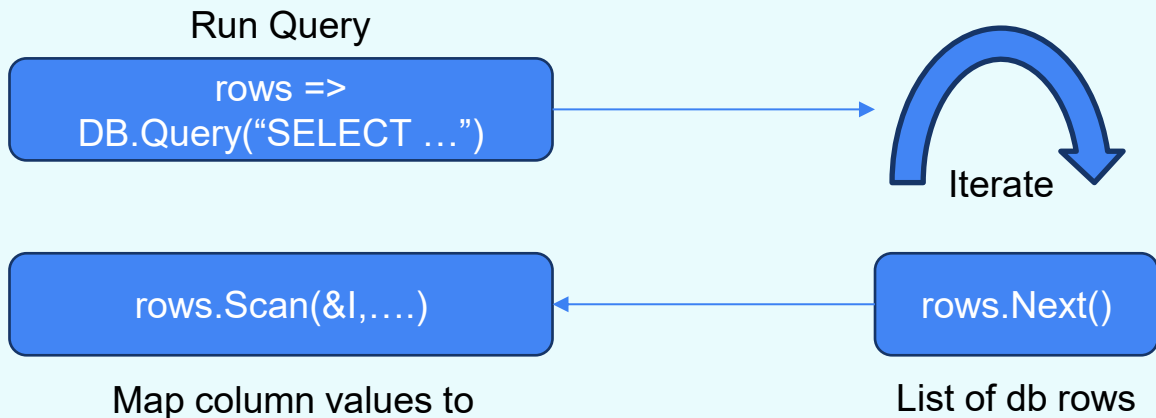
func connectDB() (*sql.DB, error) {
    connStr := "user=username dbname=mydb sslmode=disable"
    return sql.Open("postgres", connStr)
}
```



Using database package

Executing Queries?

Executing queries involves retrieving multiple rows of data with **db.Query**, iterating through **rows.Next()** to process results, and mapping column values to variables using **rows.Scan()**. Proper error handling and closing rows are crucial for efficient and safe database interactions.





Using database package

Prepared Statements?

Prepared statements improve performance and security by pre-compiling SQL queries. They prevent SQL injection and optimize query execution for repeated use.

Run Query

```
stmt =>  
DB.Prepare("INSERT INTO users(name, age) VALUES($1, $2)")
```

```
stmt.Exec("Ali", 30)
```

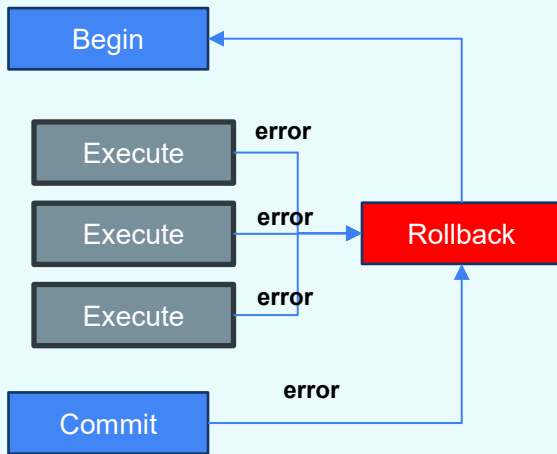




Using database package

Transactions?

Transactions ensure atomicity by grouping multiple operations into a single unit. They allow for rollback on failure, maintaining data integrity, and commit on success for reliable database operations.





Using database package

Best Practices?

Best Practice	Details
Always Close Resources	Ensure rows, statements, and transactions are closed to avoid memory leaks. Use defer for proper management.
Leverage Connection Pooling	Use sql.DB for efficient connection pooling. Avoid frequent opening and closing of connections to reduce overhead.
Validate User Inputs	Sanitize and validate inputs to prevent SQL injection. Use prepared statements or parameterized queries.
Graceful Handling of nil Results	Handle cases with no rows or null values properly. Use rows.Next() and Scan checks to prevent errors.
Use Context for Query Timeouts	Define timeouts or cancellations using context.Context to prevent resource blockage from long-running queries.
Test SQL Queries	Test SQL syntax and edge cases for reliability. Use dummy data or tools to validate complex queries.
Log Errors Clearly	Provide detailed error messages for debugging. Include relevant query details in logs.
Minimize Hardcoding	Avoid hardcoding queries or credentials in code. Use environment variables or configuration files for flexibility.
Use Transactions Wisely	Wrap critical operations in transactions for atomicity. Commit only after all queries succeed.



Using database package

Common Pitfalls?



Forgetting to check for query errors.

Not closing resources (rows, stmt, db).

Hardcoding query strings.

Ignoring context for timeout and cancellation.



Thank you. Questions?

“Learn from yesterday, live for today, hope for tomorrow. The important thing is not to stop questioning.”

- **Albert Einstein**

